VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

AN AUTONOMOUS INSTITUTION

(Accredited by NAAC A+ Approved by AICTE New Delhi & Permanently Affiliated to JNTUH)

Aziz Nagar Gate, C.B. Post, Hyderabad-500 075

DEPARTMENT OF INFORMATION TECHNOLOGY

LAB PROJECT REPORT

Roll No	:	24911A12B1
Name	:	Perka Shree Ram Kaushek
Project Title		Movie Ticket Booking System
Course Name	:	Object Oriented Programming through Java Lab
Course Code	:	A224592
Academic Year	:	2025-26
Regulations	:	R22
Year and Semester	:	II B.Tech II Semester

VIDYA JYOTHI
INSTITUTE OF TECHNOLOGY
AN AUTONOMOUS INSTITUTION

CERTIFICATE

This is to certify that the Object Oriented Programming through Java Lab project report entitled "**Movie Ticket Booking System**" submitted by **Perka Shree Ram Kaushek (24911A12B1)** to Vidya Jyothi Institute of Technology (An Autonomous Institution), Hyderabad, of B. Tech. II year, II Semester in Information Technology a Bonafide record of Lab project work carried out by them under my supervision.

Signature of Faculty	**Signature of Head of the Department**
Mr. B Eswar Babu	Dr. A. Obulesu
Associate Professor	Professor

Abstract

The **Movie Ticket Booking System** is a desktop-based application developed using Java AWT (Abstract Window Toolkit) in IntelliJ IDEA. The main purpose of this project is to provide a simple and user-friendly interface for booking movie tickets efficiently. The system allows users to browse available movies, select seats, choose a payment method, and receive a booking confirmation. All booking details are stored in a text file named **bookings.txt**, which ensures basic data persistence and record maintenance.

The need for this system arises from the limitations of manual movie ticket booking methods, which are often time-consuming, inefficient, and prone to human error. By automating the booking process, the project improves accuracy, reduces manual effort, and enhances the overall user experience. The system is designed to simulate a real-world cinema booking environment in a simplified desktop application format.

The technologies used in this project include **Java** as the programming language, **Java AWT** for building the graphical user interface, **multithreading** for implementing a real-time clock display, and **file handling** for storing and retrieving booking data. These technologies together demonstrate the practical use of core Java programming concepts in developing an interactive application.

The expected outcome of the project is a fully functional and intuitive movie ticket booking system that enables seamless ticket reservation and generates booking confirmation records. This project also highlights the real-world applicability of Java programming concepts such as event handling, object-oriented programming, multithreading, and file management

Table of Contents

Chapter 1 – Introduction

1.1 Project Overview

The **Movie Ticket Booking System** is a Java-based desktop application developed using **AWT (Abstract Window Toolkit)**. It provides a graphical user interface that enables users to select movies, choose seats, select a payment method, and receive a booking confirmation. The system is designed to simplify the ticket booking process and make it more efficient and user-friendly. All booking details are stored in a text file named **bookings.txt**, which provides basic persistence for the booking records.

This application simulates a real-world cinema ticket reservation process and can be used in movie theaters, multiplexes, and desktop-based booking kiosks. It demonstrates how Java can be used to build practical applications with graphical interfaces and data storage features.

1.2 Objectives

The main objectives of the Movie Ticket Booking System are as follows:

- To provide a graphical user interface for movie ticket booking using **Java AWT**
- To automate the process of movie selection, seat booking, and payment handling
- To implement file handling for the persistent storage of booking records
- To apply important Java concepts such as **object-oriented programming, event handling, multithreading, and file management**
- To create a simple, efficient, and user-friendly application for ticket reservation

1.3 Scope of the Project

The scope of this project is limited to a desktop-based movie ticket booking application intended for end users such as customers interacting with a cinema hall kiosk or a personal booking system. The application covers the essential functionalities of selecting a movie, booking seats, choosing a payment method, and generating a booking confirmation.

At present, the system uses file-based storage instead of a database, which makes it suitable for small-scale applications and academic demonstration purposes. In the future, the project can be enhanced by integrating a database such as **MySQL** or **SQLite**, adding user authentication, and extending the application into a web-based or mobile-based platform for broader accessibility and scalability.

Chapter 2 – System Analysis

2.1 Functional Requirements

The functional requirements of the **Movie Ticket Booking System** describe the core operations that the application must perform in order to serve the user effectively. These requirements define the main features of the system and ensure that the booking process is carried out smoothly.

- **Movie Selection:** The system should allow users to browse and select from a list of available movies.
- **Seat Booking:** The system should enable users to enter the number of seats and choose their preferred seat type.
- **Payment Method Selection:** The system should provide multiple payment options such as **Cash, Card, and UPI**.
- **Booking Confirmation:** After completing the booking process, the system should display a confirmation summary and save the booking details in **bookings.txt**.

2.2 Non-Functional Requirements

The non-functional requirements define the quality attributes of the system. These requirements ensure that the application is reliable, efficient, and easy to use.

- **Usability:** The interface should be simple, clear, and user-friendly so that even first-time users can operate the system without difficulty.
- **Performance:** The system should respond quickly to user inputs and perform booking operations without noticeable delay.
- **Reliability:** Booking data should be stored and retrieved accurately from the file without data loss or corruption.
- **Maintainability:** The code should be easy to understand and modify, allowing future improvements and enhancements.
- **Portability:** The application should be capable of running on different operating systems such as **Windows, macOS, and Linux**.

2.3 Software Requirements

The software requirements for developing and running the Movie Ticket Booking System are listed below:

- **Operating System:** Windows / macOS / Linux
- **Programming Language:** Java
- **IDE:** IntelliJ IDEA
- **Storage:** File System (**bookings.txt**)
- **JDK:** JDK 8 or above

2.4 Hardware Requirements

The hardware requirements for executing the application effectively are as follows:

- **RAM:** 4 GB
- **Processor:** Intel i3 or above
- **Storage:** 500 GB

Chapter 3 – System Design

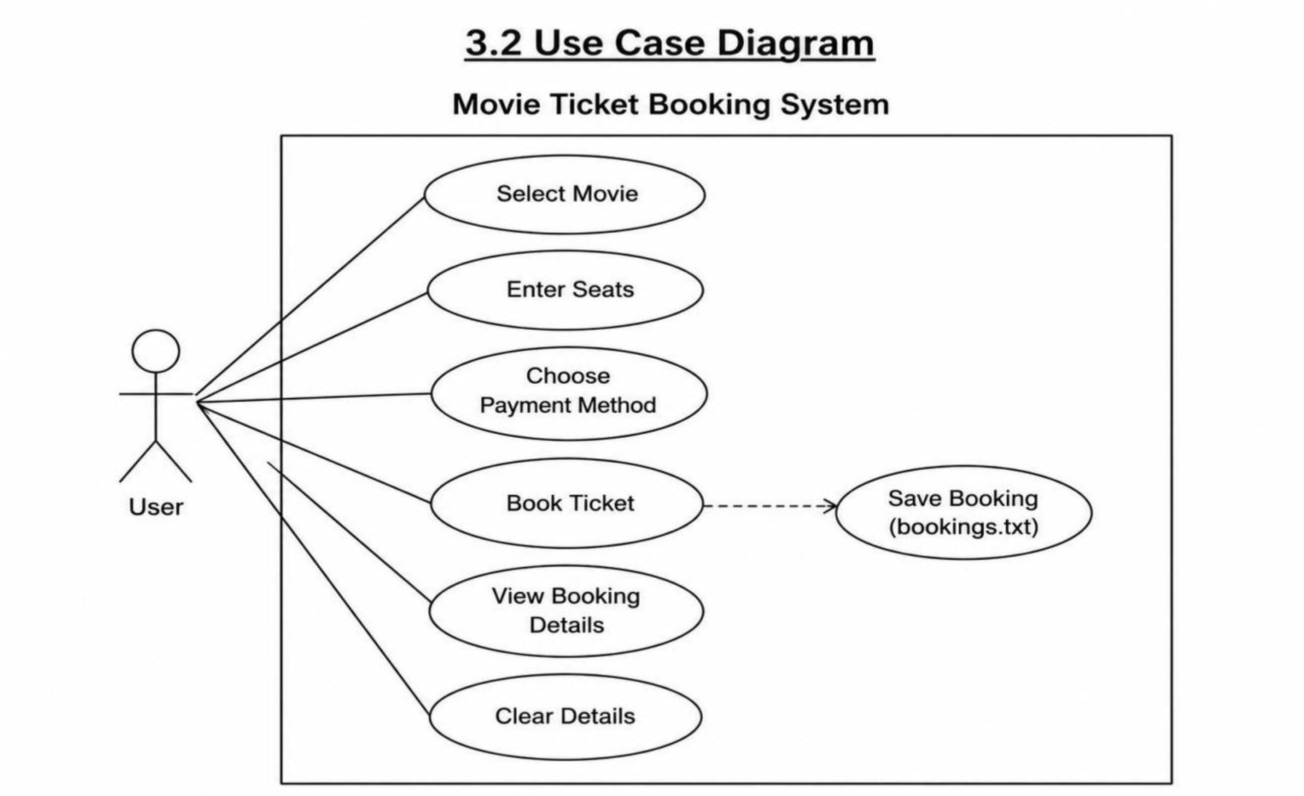
3.1 Architecture Diagram

Simple block diagram.

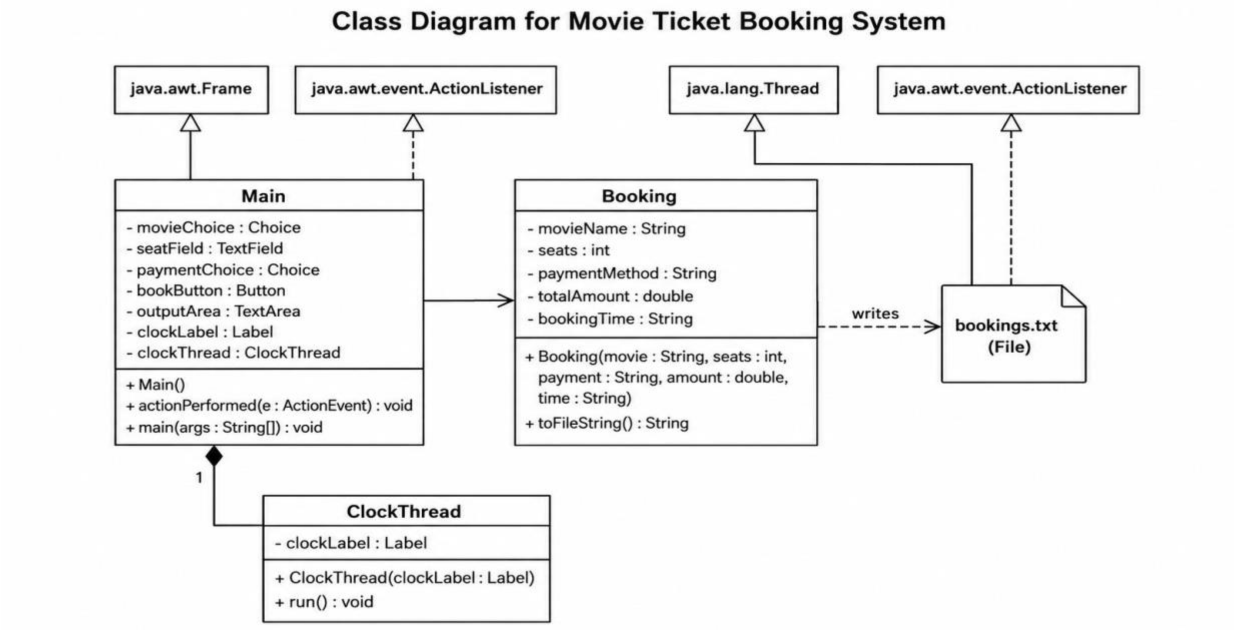
User → Java AWT GUI → Booking Classes → bookings.txt File

3.2 UML Diagrams

- **Use Case Diagram**



- **Class Diagram**



3.3 Database Design

Table Name	Purpose
users	Store customer login details
movies	Store movie information
shows	Store movie show timings
bookings	Store ticket booking details
payments	Store payment records
admins	Store admin login details
seats	Store seat availability information

Chapter 4 – Implementation

4.1 Modules Description

Module Description

Movie Selection Module Displays available movies using Choice and allows the user to select a movie.

Seat Booking Module Allows the user to enter the number of seats for booking.

Payment Module Handles payment method selection and calculates the total booking amount.

Booking Confirmation Module Displays booking status and stores the booking details in bookings.txt.

4.2 Important Java Concepts Used

The following Java syllabus concepts were applied in this project:

- **Classes & Objects** – The project uses classes such as Booking, ClockThread, and Main to organize data and functionality.
- **Inheritance** – The Main class extends Frame, and ClockThread extends Thread.
- **Event Handling** – The application implements ActionListener and uses actionPerformed() to respond to button clicks.
- **Constructor Usage** – Constructors are used in Booking, ClockThread, and Main to initialize objects.
- **Exception Handling** – try-catch blocks handle invalid seat input and other runtime errors.
- **Multithreading** – A separate thread updates the live clock every second.
- **File Handling** – FileWriter and BufferedWriter are used to store booking details in bookings.txt.
- **AWT** – GUI components such as Frame, Label, Choice, TextField, Button, and TextArea are used from java.awt.

4.3 Code Snippets

This section includes important code snippets from the project, such as the Booking class for storing reservation details, the ClockThread class for displaying the live time, the GUI setup in Main, ticket price calculation based on movie selection, input validation for seat count, and file writing logic used to save booking details into bookings.txt.

Booking.java

```
class Booking {
  String movieName;
  int seatCount;

  Booking(String movieName, int seatCount) {
    this.movieName = movieName;
    this.seatCount = seatCount;
  }

  public String getDetails() {
    return "Movie: " + movieName + " | Seats: " + seatCount;
  }
}
```

ClockThread.java

```
class ClockThread extends Thread {
  Label clockLabel;

  ClockThread(Label clockLabel) {
    this.clockLabel = clockLabel;
  }

  public void run() {
```

```
    while (true) {
      String time = new SimpleDateFormat("hh:mm:ss a").format(new Date());
      clockLabel.setText("Current Time: " + time);
      try { Thread.sleep(1000); } catch (Exception e) { System.out.println(e); }
    }
  }
}
```

Main.java (Booking and File Writing Logic)

```
String movie = movieChoice.getSelectedItem();
String payment = paymentChoice.getSelectedItem();
int seats = Integer.parseInt(seatField.getText());

if (seats <= 0) {
  throw new Exception("Invalid seat count");
}

Booking booking = new Booking(movie, seats);
outputArea.setText("Ticket Booked Successfully\n\n");
outputArea.append(booking.getDetails());
outputArea.append("\nPayment Method: " + payment);

FileWriter fw = new FileWriter("bookings.txt", true);
BufferedWriter bw = new BufferedWriter(fw);
bw.write(booking.getDetails());
bw.write(" | Payment: " + payment);
bw.newLine();
bw.close();
```

Chapter 5 – Testing

5.1 Test Cases

Test Case ID	Input	Expected Output	Result
TC01	Select a movie and enter 2 seats	Booking confirmation displayed and saved to bookings.txt	Pass
TC02	Enter 0 seats	Error: Please enter a valid number of seats	Pass

5.2 Error Handling

The following exceptions are handled in the application:

NumberFormatException – Caught when non-numeric input is entered for seat count.

IOException – Caught when bookings.txt cannot be opened or written to.

IllegalArgumentException – Caught when an invalid payment method is selected.

Chapter 6 – Results and Screenshots

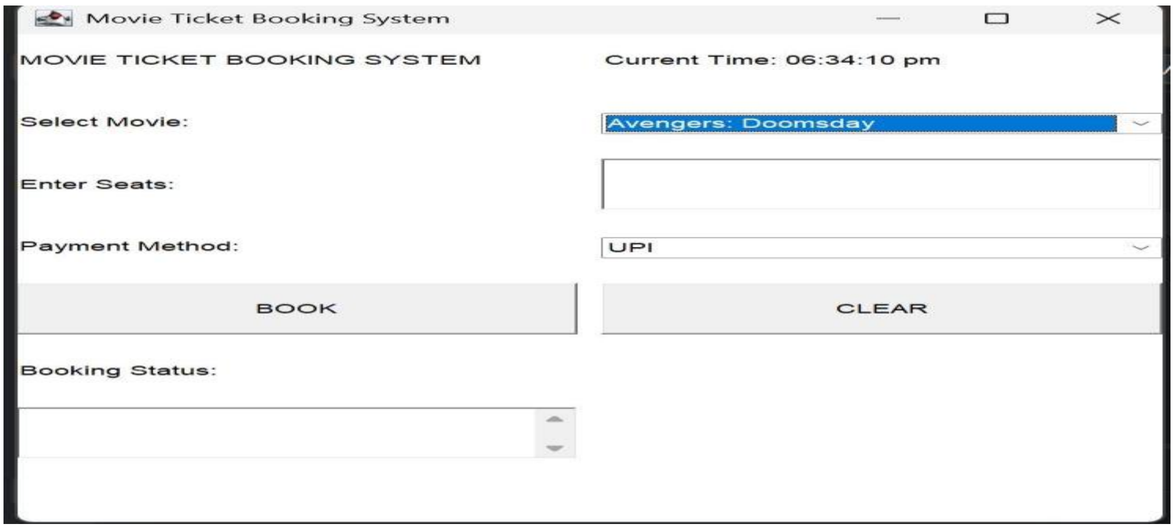


Figure 1: Movie Selection Screen

Description: Displays the list of available movies for selection.

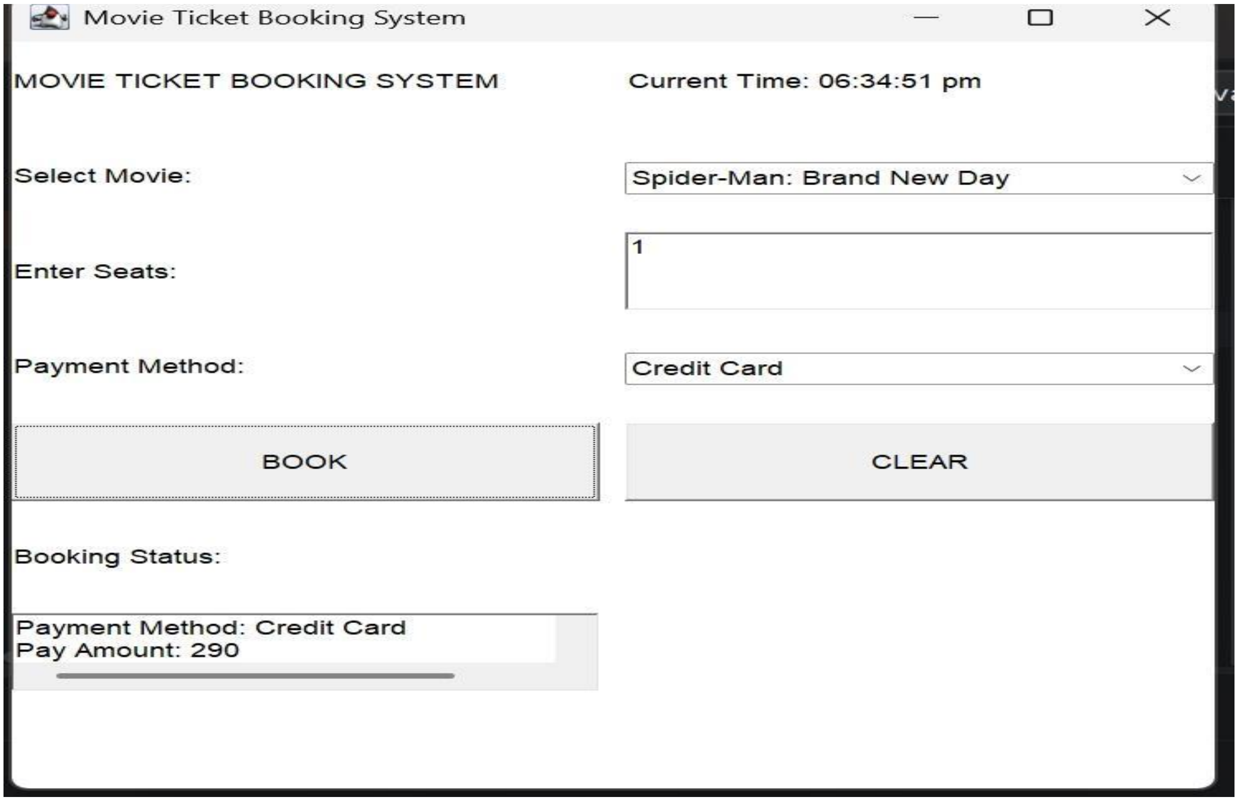


Figure 2: Payment Screen

Description: Displays the payment options and total booking cost.

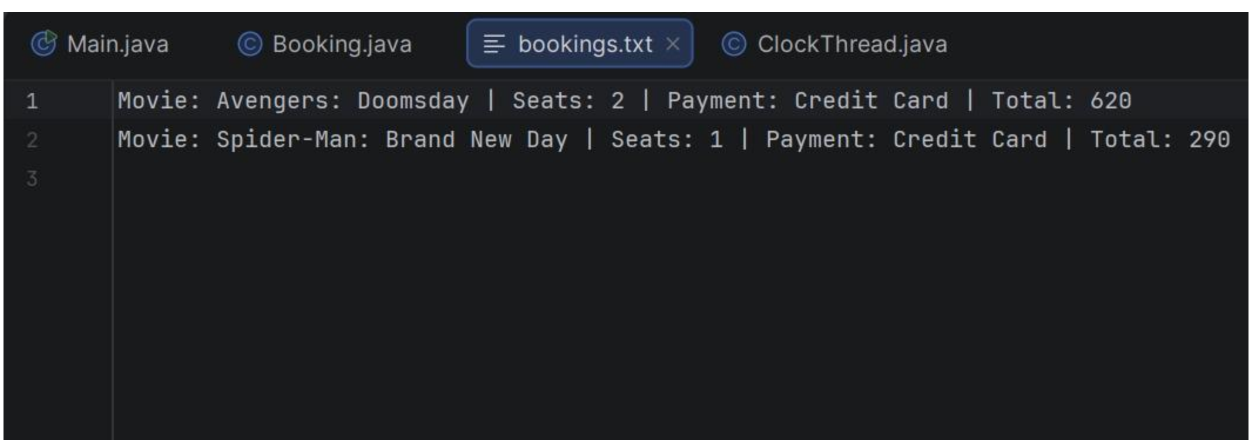


Figure 3: bookings.txt Output

Description: Displays the saved booking record in the output file

Chapter 7 – Conclusion and Future Scope

Conclusion

The **Movie Ticket Booking System** was successfully developed and tested. The project demonstrates practical application of Java AWT, file I/O, multithreading, and object-oriented programming concepts in building a user-friendly desktop application. Key learning outcomes include building GUI applications, handling user events, and managing persistent data without a database.

Future Enhancements

- Database integration – Replace file storage with MySQL or SQLite for scalability.
- Online booking portal – Convert the system into a web application using Java Servlets or Spring Boot.
- Payment gateway – Integrate UPI and card payment APIs for real transactions.

References

References used in this project are listed below:

1. Herbert Schildt, *Java: The Complete Reference*, 12th Edition, McGraw-Hill, 2021.
2. Bruce Eckel, *Thinking in Java*, 4th Edition, Prentice Hall, 2006.
3. Oracle Corporation, Java AWT Documentation.
4. GeeksforGeeks, Java File Handling Tutorial.